

os codigos do Copilot
que deram erros:

```
# =====
# PROJETO ACADÊMICO
# SISTEMA DE ANÁLISE E OTIMIZAÇÃO DE TRELIÇAS PLANAS ISOSTÁTICAS
# =====
# Autor: Licarmilo
# Disciplina: Estruturas de Engenharia Civil
# Linguagem: Python
# Bibliotecas utilizadas: math, numpy, matplotlib
#
# Descrição:
# Este programa realiza a análise completa de uma treliça plana isostática.
# Ele calcula automaticamente:
# - Reações de apoio
# - Forças internas nas barras
# - Classificação das barras (tração, compressão, barra nula)
# - Barra mais solicitada
# - Força média
# - Somatório das reações
#
# Além disso, gera gráficos:
# (1) Treliça original com nós e apoios
# (2) Treliça colorida por intensidade de força
# (3) Gráfico de barras das forças internas
# (4) Histograma da distribuição das forças
#
# =====

import math
import numpy as np
import matplotlib.pyplot as plt

# =====
# FUNÇÕES AUXILIARES
# =====

def gauss_jordan(A, b):
    """
    Resolve sistema linear  $Ax = b$  pelo método de Gauss-Jordan.
    Implementação manual sem uso de bibliotecas de álgebra linear.
    """
    n = len(A)
    # Construir matriz aumentada
    M = [A[i] + [b[i]] for i in range(n)]

    for i in range(n):
```

```

# Pivoteamento
if M[i][i] == 0:
    for k in range(i+1, n):
        if M[k][i] != 0:
            M[i], M[k] = M[k], M[i]
            break

# Normalizar pivô
pivot = M[i][i]
if pivot == 0:
    continue
for j in range(i, n+1):
    M[i][j] /= pivot

# Eliminar outras linhas
for k in range(n):
    if k != i:
        factor = M[k][i]
        for j in range(i, n+1):
            M[k][j] -= factor * M[i][j]

# Extrair solução
x = [M[i][n] for i in range(n)]
return x

def distancia(p1, p2):
    """Calcula distância entre dois pontos."""
    return math.sqrt((p2[0]-p1[0])**2 + (p2[1]-p1[1])**2)

def direcao(p1, p2):
    """Calcula cosseno e seno da direção da barra."""
    L = distancia(p1, p2)
    cx = (p2[0]-p1[0]) / L
    cy = (p2[1]-p1[1]) / L
    return cx, cy

# =====
# DADOS DA TRELIÇA (EXEMPLO REALISTA)
# =====

# Coordenadas dos nós (x,y)
nos = {
    1: (0,0),
    2: (2,0),
    3: (4,0),
    4: (6,0),
    5: (1,2),
    6: (3,2),

```

```
7: (5,2),
8: (7,2)
}
```

```
# Barras (nó inicial, nó final)
```

```
barras = {
    1: (1,2),
    2: (2,3),
    3: (3,4),
    4: (1,5),
    5: (2,5),
    6: (2,6),
    7: (3,6),
    8: (3,7),
    9: (4,7),
    10: (5,6),
    11: (6,7),
    12: (7,8),
    13: (5,7),
    14: (6,8)
}
```

```
# Apoios: nó -> tipo ("fixo" = engaste, "móvel" = rolete vertical)
```

```
apoios = {
    1: "fixo",
    4: "móvel"
}
```

```
# Cargas aplicadas: nó -> (Fx, Fy)
```

```
cargas = {
    5: (0,-10),
    6: (0,-15),
    7: (0,-10),
    8: (0,-5)
}
```

```
# =====
# MONTAGEM DO SISTEMA DE EQUAÇÕES
# =====
```

```
# Cada nó tem duas equações (x e y)
```

```
n_equacoes = len(nos)*2
```

```
n_incognitas = len(barras) + sum([2 if apoios[n]=="fixo" else 1 for n in apoios])
```

```
A = [[0]*n_incognitas for _ in range(n_equacoes)]
```

```
b = [0]*n_equacoes
```

```
# Mapear incógnitas: barras + reações
```

```

mapa_incognitas = {}
idx = 0
for barra in barras:
    mapa_incognitas[f"B{barra}"] = idx
    idx += 1
for no in apoios:
    if apoios[no] == "fixo":
        mapa_incognitas[f"Rx{no}"] = idx; idx+=1
        mapa_incognitas[f"Ry{no}"] = idx; idx+=1
    elif apoios[no] == "móvel":
        mapa_incognitas[f"Ry{no}"] = idx; idx+=1

# Montar equações de equilíbrio em cada nó
for no in nos:
    eqx = (no-1)*2
    eqy = eqx+1
    # Forças das barras conectadas
    for barra, (n1,n2) in barras.items():
        if no in (n1,n2):
            cx, cy = direcao(nos[n1], nos[n2])
            sinal = 1 if no==n1 else -1
            A[eqx][mapa_incognitas[f"B{barra}"]] = sinal*cx
            A[eqy][mapa_incognitas[f"B{barra}"]] = sinal*cy
    # Reações de apoio
    if no in apoios:
        if apoios[no]=="fixo":
            A[eqx][mapa_incognitas[f"Rx{no}"]] = 1
            A[eqy][mapa_incognitas[f"Ry{no}"]] = 1
        elif apoios[no]=="móvel":
            A[eqy][mapa_incognitas[f"Ry{no}"]] = 1
    # Cargas externas
    if no in cargas:
        b[eqx] = -cargas[no][0]
        b[eqy] = -cargas[no][1]

# =====
# RESOLUÇÃO DO SISTEMA
# =====

solucao = gauss_jordan(A,b)

# Separar resultados
forcas_barras = {}
for barra in barras:
    forcas_barras[barra] = solucao[mapa_incognitas[f"B{barra}"]]

reacoes = {}
for no in apoios:

```

```

if apoios[no]=="fixo":
    reacoes[f"Rx{no}"] = solucao[mapa_incognitas[f"Rx{no}"]]
    reacoes[f"Ry{no}"] = solucao[mapa_incognitas[f"Ry{no}"]]
elif apoios[no]=="móvel":
    reacoes[f"Ry{no}"] = solucao[mapa_incognitas[f"Ry{no}"]]

# =====
# CLASSIFICAÇÃO DAS BARRAS
# =====

classificacao = {}
for barra, F in forcas_barras.items():
    if abs(F) < 1e-6:
        classificacao[barra] = "Nula"
    elif F > 0:
        classificacao[barra] = "Tração"
    else:
        classificacao[barra] = "Compressão"

# =====
# RESULTADOS NUMÉRICOS
# =====

print("\n===== RESULTADOS =====\n")
print("Forças internas nas barras:")
print("-----")
for barra in barras:
    print(f"Barra {barra:2d}: {forcas_barras[barra]:8.3f} kN -> {classificacao[barra]}")

print("\nReações de apoio:")
print("-----")
for r in reacoes:
    print(f"{r}: {reacoes[r]:8.3f} kN")

# Barra mais solicitada
barra_max = max(forcas_barras, key=lambda k: abs(forcas_barras[k]))
print("\nBarra mais solicitada:")
print(f"Barra {barra_max} com {forcas_barras[barra_max]:.3f} kN ({classificacao[barra_max]}")

# Força média
media = sum(abs(f) for f in forcas_barras.values())/len(forcas_barras)
print(f"\nForça média nas barras: {media:.3f} kN")

# Somatório das reações
somatorio_reacoes = sum(reacoes.values())
print(f"\nSomatório das reações: {somatorio_reacoes:.3f} kN")

```

```
# =====  
# GR
```

codigo depois da correção

```
# =====  
# PROJETO ACADÊMICO  
# SISTEMA DE ANÁLISE E OTIMIZAÇÃO DE TRELIÇAS PLANAS ISOSTÁTICAS  
# =====  
# Autor: Licarmilo  
# =====
```

```
import math  
import matplotlib.pyplot as plt
```

```
# =====  
# FUNÇÕES AUXILIARES  
# =====
```

```
def gauss_jordan(A, b):  
    """Resolve sistema linear Ax = b pelo método de Gauss-Jordan."""  
    n = len(A)  
    M = [A[i] + [b[i]] for i in range(n)]  
  
    for i in range(n):  
        if abs(M[i][i]) < 1e-12:  
            for k in range(i+1, n):  
                if abs(M[k][i]) > 1e-12:  
                    M[i], M[k] = M[k], M[i]  
                    break  
        pivot = M[i][i]  
        if abs(pivot) < 1e-12:  
            continue  
        for j in range(i, n+1):  
            M[i][j] /= pivot  
        for k in range(n):  
            if k != i:  
                factor = M[k][i]  
                for j in range(i, n+1):  
                    M[k][j] -= factor * M[i][j]  
  
    return [M[i][n] for i in range(n)]  
  
def distancia(p1, p2):  
    return math.sqrt((p2[0]-p1[0])**2 + (p2[1]-p1[1])**2)
```

```

def direcao(p1, p2):
    L = distancia(p1, p2)
    return (p2[0]-p1[0])/L, (p2[1]-p1[1])/L

# =====
# DADOS DA TRELIÇA ISOSTÁTICA
# =====

nos = {
    1: (0,0), 2: (2,0), 3: (4,0), 4: (6,0),
    5: (1,2), 6: (3,2), 7: (5,2), 8: (7,2)
}

# 13 barras (isostática: m=2j-r = 13)
barras = {
    1:(1,2),2:(2,3),3:(3,4),4:(1,5),5:(2,5),6:(2,6),
    7:(3,6),8:(3,7),9:(4,7),10:(5,6),11:(6,7),
    12:(7,8),13:(5,7)
}

apoios = {1:"fixo",4:"móvel"}
cargas = {5:(0,-10),6:(0,-15),7:(0,-10),8:(0,-5)}

# =====
# MONTAGEM DO SISTEMA
# =====

n_equacoes = len(nos)*2
n_incognitas = len(barras)
for no in apoios:
    n_incognitas += 2 if apoios[no]=="fixo" else 1

A = [[0]*n_incognitas for _ in range(n_equacoes)]
b = [0]*n_equacoes

mapa_incognitas = {}
idx=0
for barra in barras:
    mapa_incognitas[f"B{barra}"]=idx; idx+=1
for no in apoios:
    if apoios[no]=="fixo":
        mapa_incognitas[f"Rx{no}"]=idx; idx+=1
        mapa_incognitas[f"Ry{no}"]=idx; idx+=1
    else:
        mapa_incognitas[f"Ry{no}"]=idx; idx+=1

for no in nos:
    eqx=(no-1)*2; eqy=eqx+1

```

```

for barra,(n1,n2) in barras.items():
    if no in (n1,n2):
        cx,cy=direcao(nos[n1],nos[n2])
        sinal=1 if no==n1 else -1
        A[eqx][mapa_incognitas[f"B{barra}"]]=sinal*cx
        A[eqy][mapa_incognitas[f"B{barra}"]]=sinal*cy
    if no in apoios:
        if apoios[no]=="fixo":
            A[eqx][mapa_incognitas[f"Rx{no}"]]=1
            A[eqy][mapa_incognitas[f"Ry{no}"]]=1
        else:
            A[eqy][mapa_incognitas[f"Ry{no}"]]=1
    if no in cargas:
        b[eqx]=-cargas[no][0]; b[eqy]=-cargas[no][1]

# =====
# RESOLUÇÃO
# =====

solucao=gauss_jordan(A,b)
forcas_barras={barra:solucao[mapa_incognitas[f"B{barra}"]] for barra in barras}
reacoes={}
for no in apoios:
    if apoios[no]=="fixo":
        reacoes[f"Rx{no}"]=solucao[mapa_incognitas[f"Rx{no}"]]
        reacoes[f"Ry{no}"]=solucao[mapa_incognitas[f"Ry{no}"]]
    else:
        reacoes[f"Ry{no}"]=solucao[mapa_incognitas[f"Ry{no}"]]

classificacao={}
for barra,F in forcas_barras.items():
    if abs(F)<1e-6: classificacao[barra]="Nula"
    elif F>0: classificacao[barra]="Tração"
    else: classificacao[barra]="Compressão"

# =====
# RESULTADOS
# =====

print("\n===== RESULTADOS =====\n")
print("Forças internas nas barras:")
for barra in barras:
    print(f"Barra {barra:2d}: {forcas_barras[barra]:8.3f} kN -> {classificacao[barra]}")

print("\nReações de apoio:")
for r in reacoes:
    print(f"{r}: {reacoes[r]:8.3f} kN")

```



```

barra_max=max(forcas_barras,key=lambda k:abs(forcas_barras[k]))
print(f"\nBarra mais solicitada: {barra_max} ({classificacao[barra_max]}) com
{forcas_barras[barra_max]:.3f} kN")
media=sum(abs(f) for f in forcas_barras.values())/len(forcas_barras)
print(f"Força média nas barras: {media:.3f} kN")
print(f"Somatório das reações: {sum(reacoes.values()):.3f} kN")

# =====
# GRÁFICOS
# =====

# (1) Treliza original
plt.figure(figsize=(10,6))
for barra,(n1,n2) in barras.items():
    x=[nos[n1][0],nos[n2][0]]; y=[nos[n1][1],nos[n2][1]]
    plt.plot(x,y,'k-',linewidth=1)
for no,(x,y) in nos.items():
    plt.plot(x,y,'bo'); plt.text(x+0.05,y+0.05,str(no))
for no,tipo in apoios.items():
    x,y=nos[no]
    if tipo=="fixo": plt.plot(x,y,'gs',markersize=10,label="Apoio fixo" if no==1 else "")
    else: plt.plot(x,y,'r^',markersize=10,label="Apoio móvel" if no==4 else "")
plt.title("Treliza Original"); plt.legend(); plt.axis('equal'); plt.show()

# (2) Treliza colorida por intensidade
plt.figure(figsize=(10,6))
for barra,(n1,n2) in barras.items():
    x=[nos[n1][0],nos[n2][0]]; y=[nos[n1][1],nos[n2][1]]
    F=forcas_barras[barra]
    cor='blue' if F>0 else ('red' if F<0 else 'gray')
    plt.plot(x,y,color=cor,linewidth=2)
for no,(x,y) in nos.items():
    plt.plot(x,y,'ko'); plt.text(x+0.05,y+0.05,str(no))
plt.title("Treliza colorida por intensidade de força")
plt.axis('equal'); plt.show()

# (3) Gráfico de barras
plt.figure(figsize=(10,6))
plt.bar(list(forcas_barras.keys()),list(forcas_barras.values()),color='skyblue')
plt.title("Força em cada barra"); plt.xlabel("Barra"); plt.ylabel("Força (kN)")
plt.show()

# (4) Histograma
plt.figure(figsize=(10,6))
plt.hist(list(forcas_barras.values()), bins=10, color='orange', edgecolor='black')
plt.title("Distribuição das forças internas")
plt.xlabel

```